

Contents

Chapter 0 — Foundations

- 0.1 What is a language model?
- 0.2 n-gram models (the statistical era)
- 0.3 Feedforward Neural Network LM (Bengio, 2003)
- 0.4 Word embeddings (Word2Vec, GloVe)
- 0.5 Activation functions (the non-linearities used everywhere)
- 0.6 The scoreboard entering the neural sequence era

Chapter 1 — Recurrent Neural Networks (RNN)

- 1.1 What it is
- 1.2 Why it appeared (the limitation it fixed)
- 1.3 Complete architecture
- 1.4 How it does language modelling (step by step)
- 1.5 Training — Backpropagation Through Time (BPTT)
- 1.6 Limitations — vanishing and exploding gradients
- 1.7 How it gave rise to the next model

Chapter 2 — LSTM & GRU

- 2.1 What it is
- 2.2 Why it appeared (the limitation it fixed)
- 2.3 Complete architecture
- 2.4 How it works — the update, step by step
- 2.5 The GRU — a simpler alternative
- 2.6 Limitations
- 2.7 How it gave rise to the next model

Chapter 3 — Sequence-to-Sequence (Seq2Seq)

- 3.1 What it is
- 3.2 Why it appeared (the limitation it fixed)
- 3.3 Complete architecture

3.4 How it works — step by step

3.5 Limitations — the fixed-vector bottleneck

3.6 How it gave rise to the next model

Chapter 4 — Attention

4.1 What it is

4.2 Why it appeared (the limitation it fixed)

4.3 The core mechanism — Query, Key, Value

4.4 Scoring functions

4.5 How attention changes Seq2Seq

4.6 Self-attention — the pivotal generalization

4.7 Limitations of attention added onto RNNs

4.8 How it gave rise to the next model

Chapter 5 — The Transformer

5.1 What it is

5.2 Why it appeared (the limitation it fixed)

5.3 Complete architecture

5.4 Component 1 — Input embeddings + Positional Encoding

5.5 Component 2 — Scaled Dot-Product Attention

5.6 Component 3 — Multi-Head Attention

5.7 Component 4 — Position-wise Feedforward Network

5.8 Component 5 — Residual connections + Layer Normalization

5.9 Encoder vs. Decoder — and the three kinds of attention

5.10 How the Transformer does language modelling — full data flow

5.11 Limitations

5.12 How it gave rise to the next models

Chapter 6 — BERT

6.1 What it is

6.2 Why it appeared (the limitation it fixed)

6.3 Complete architecture

6.4 Pre-training task 1 — Masked Language Modelling (MLM)

6.5 Pre-training task 2 — Next Sentence Prediction (NSP)

6.6 The pre-train → fine-tune paradigm

6.7 Limitations

6.8 How it relates to the next model

Chapter 7 — GPT

7.1 What it is

7.2 Why it appeared (the limitation it fixed)

7.3 Complete architecture

7.4 How it does language modelling

7.5 The paradigm shift — from fine-tuning to prompting

7.6 From raw model to assistant — instruction tuning and RLHF

7.7 Limitations

7.8 How it gave rise to what comes next

Chapter 8 — Beyond: Modern LLMs

8.1 The three Transformer families

8.2 Key models and ideas after BERT/GPT

8.3 Architectural refinements you will encounter

8.4 Tackling the quadratic-attention limitation

8.5 The full evolution, at a glance

8.6 The recurring pattern of every leap

8.7 A compact glossary

The Evolution of Language Models — A Complete Study Guide

Reading order

The one-picture summary

The single thread that connects everything

Chapter 0 — Foundations

Before RNNs, "language modelling" already existed. Understanding the pre-neural and early-neural methods makes it obvious *why* every later architecture was invented.

0.1 What is a language model?

A **language model (LM)** assigns a probability to a sequence of words, or equivalently predicts the next word given the words so far.

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

If a model can estimate $P(w_t \mid \text{context})$ well, it can:

- **Score** sentences ("the cat sat" is more likely than "the cat sats").
- **Generate** text by sampling the next word repeatedly.
- **Represent** meaning, because predicting words well requires understanding them.

Everything in this guide is a different machine for computing that one conditional probability.

0.2 n-gram models (the statistical era)

The oldest practical LM. It makes a **Markov assumption**: the next word depends only on the previous $n - 1$ words.

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1})$$

Probabilities are just **counts** from a text corpus:

$$P(w_t \mid w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

Example (bigram, n=2): to predict the word after "New", count how often each word follows "New" in the corpus. "York" wins.

Limitations of n-grams

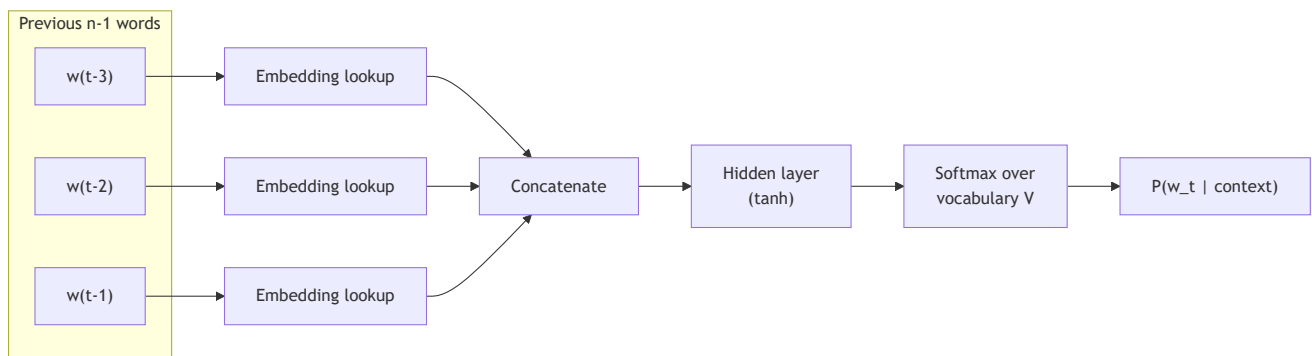
Limitation	Consequence
Fixed, tiny context	A trigram cannot use anything older than 2 words back.

Limitation	Consequence
Data sparsity	Most valid n-grams never appear in training, giving probability 0 (patched with smoothing/back-off).
No notion of similarity	"cat" and "dog" are unrelated symbols; learning about one teaches nothing about the other.
Exponential table growth	Vocabulary V with n-grams needs up to V^n counts.

These four problems are exactly what neural models set out to fix.

0.3 Feedforward Neural Network LM (Bengio, 2003)

The first neural LM replaced the count table with a small neural network. It still uses a **fixed window** of the previous $n - 1$ words, but each word is mapped to a learned dense vector (an **embedding**) instead of a one-hot symbol.



Data flow:

1. Look up an embedding vector for each of the previous $n - 1$ words.
2. Concatenate them into one long vector x .
3. Pass through a hidden layer: $h = \tanh(Wx + b)$.
4. Project to vocabulary size and apply softmax: $\hat{y} = \text{softmax}(Uh + d)$.
5. $\hat{y}$ is a probability distribution over the next word.

What this fixed: words now live in a continuous space, so "cat" and "dog" end up with similar vectors. Learning about one word generalizes to similar words — solving n-gram sparsity and the "no similarity" problem.

What it did not fix: the context is *still a fixed window*. It cannot handle arbitrary-length history. That is the door RNNs walk through.

0.4 Word embeddings (Word2Vec, GloVe)

A refinement of the "words as vectors" idea, trained specifically to produce high-quality vectors that can be reused everywhere.

- **Word2Vec (2013)** trains a shallow network on a simple task: predict a word from its neighbours (**CBOW**) or predict neighbours from a word (**Skip-gram**).
- **GloVe (2014)** factorizes a global word co-occurrence matrix.

The famous property that emerges:

$$\text{vec}(\text{king}) - \text{vec}(\text{man}) + \text{vec}(\text{woman}) \approx \text{vec}(\text{queen})$$

Key limitation — embeddings are static. Each word gets exactly one vector, so "bank" (river) and "bank" (money) share a vector. Context cannot change the meaning. Producing **contextual** representations is a major motivation for RNNs, and later the whole point of BERT and GPT.

0.5 Activation functions (the non-linearities used everywhere)

Every model in this guide is built from linear operations ($Wx + b$) stacked together. But a stack of linear operations is *itself just one linear operation* — no matter how many layers you add, it can only draw straight-line decision boundaries. **Activation functions** are the small non-linear functions applied after a linear layer; they are what let neural networks approximate complex, curved functions and therefore learn the rich patterns of language. This section defines each activation you will meet in later chapters, its math, and why it is used.

Sigmoid (logistic)

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- **Shape / output range:** an S-curve that squashes any real number into (0, 1).
- **Why it is used:** the output looks like a **probability** or a **soft on/off switch**. This is exactly why LSTM and GRU **gates** use it — a gate value near 1 means "let this through," near 0 means "block it." It is also used for binary classification outputs.
- **Derivative:** $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, which is at most 0.25.
- **Weakness:** for large $|x|$ the curve is flat, so its gradient is nearly 0 (**saturation**). Multiplying many such small gradients is a direct cause of the **vanishing gradient problem** in RNNs (Chapter 1).

Tanh (hyperbolic tangent)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- **Shape / output range:** an S-curve like sigmoid but squashing into $(-1, 1)$.
- **Why it is used:** it is **zero-centred**, so outputs can be positive or negative. This keeps the average signal near zero, which makes optimization more stable and faster than sigmoid. It is the default for producing a **hidden state / candidate value** in RNNs and LSTMs (the "how much and in which direction" of memory).
- **Derivative:** $\tanh'(x) = 1 - \tanh^2(x)$, at most 1 (larger than sigmoid's, but still saturates toward 0 for large $|x|$ — so $\tanh$ also contributes to vanishing gradients over long sequences).

ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x)$$

- **Shape / output range:** identity for positive inputs, flat zero for negatives; range $[0, \infty)$.
- **Why it is used:** it is **cheap** (a single comparison) and, crucially, its gradient is exactly **1** for positive inputs — it does **not saturate** on the positive side, so gradients flow well through deep networks. This is why it is the standard activation in the **feedforward sub-layers of the Transformer** (Chapter 5) and most deep nets.
- **Derivative:** 1 if $x > 0$, else 0.
- **Weakness:** the "**dying ReLU**" problem — neurons stuck at negative inputs output 0 forever and stop learning. Variants fix this: **Leaky ReLU** ($\max(0.01x, x)$) and **GELU** (below).

GELU (Gaussian Error Linear Unit)

$$\text{GELU}(x) = x \cdot \Phi(x)$$

where $\Phi(x)$ is the cumulative distribution function of the standard normal.

- **Why it is used:** a smooth, curved version of ReLU that weights inputs by their value rather than hard-gating at 0. This smoothness helps training, and it is the activation used inside **BERT and GPT** feedforward blocks.
- **Relation to ReLU:** behaves almost like ReLU for large $|x|$ but transitions gently near 0.

Softmax (the output layer)

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^V e^{z_j}}$$

- **Shape / output:** converts a vector of raw scores (**logits**) into a **probability distribution** — all outputs are positive and sum to 1.
- **Why it is used:** language models must output $P(\text{next word})$ over a vocabulary of size V . Softmax is the final layer that turns the network's scores into those probabilities. It also appears **inside attention** (Chapters 4–5) to turn relevance scores into weights that sum to 1.
- **Note:** the exponential exaggerates differences — the largest logit dominates — which is why a **temperature** parameter is used at generation time (Chapter 7) to make the distribution sharper or

flatter.

Quick comparison

Function	Formula	Output range	Primary use in this guide
Sigmoid	$\frac{1}{1 + e^{-x}}$	(0, 1)	LSTM/GRU gates; binary outputs
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	(-1, 1)	RNN/LSTM hidden state & candidates
ReLU	$\max(0, x)$	$[0, \infty)$	Transformer feedforward layers
GELU	$x \Phi(x)$	$\approx [-0.17, \infty)$	BERT / GPT feedforward layers
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	(0, 1), sums to 1	Final next-word layer; attention weights

The through-line: *saturating* activations (sigmoid, tanh) are great for gating and bounded states but cause vanishing gradients, which motivates the LSTM (Chapter 2); *non-saturating* activations (ReLU, GELU) keep gradients healthy in the very deep Transformer stacks (Chapters 5–7); and *softmax* is the constant that turns scores into probabilities, whether for the next word or for attention.

0.6 The scoreboard entering the neural sequence era

Capability	n-gram	FF-NNLM	Word2Vec
Word similarity / generalization	X	✓	✓
Arbitrary-length context	X	X	X
Contextual (word meaning changes with sentence)	X	X	X
Sequential order modelling	partial	fixed window	X

The next big leap is a network that reads a sequence **one token at a time** and keeps a **memory** of everything seen so far. That is the Recurrent Neural Network.

➔ Continue to [Chapter 1 — RNN](#)

Chapter 1 — Recurrent Neural Networks (RNN)

1.1 What it is

A **Recurrent Neural Network** is a network that reads a sequence **one element at a time** and maintains a **hidden state** — a vector that acts as a running memory of everything seen so far. At each step it combines the current input with the previous memory to produce a new memory.

The core idea in one line:

$$h_t = f(h_{t-1}, x_t)$$

The new memory h_t depends on the previous memory h_{t-1} and the current input x_t . Because the same function f (the same weights) is applied at every step, an RNN can process a sequence of **any length**.

1.2 Why it appeared (the limitation it fixed)

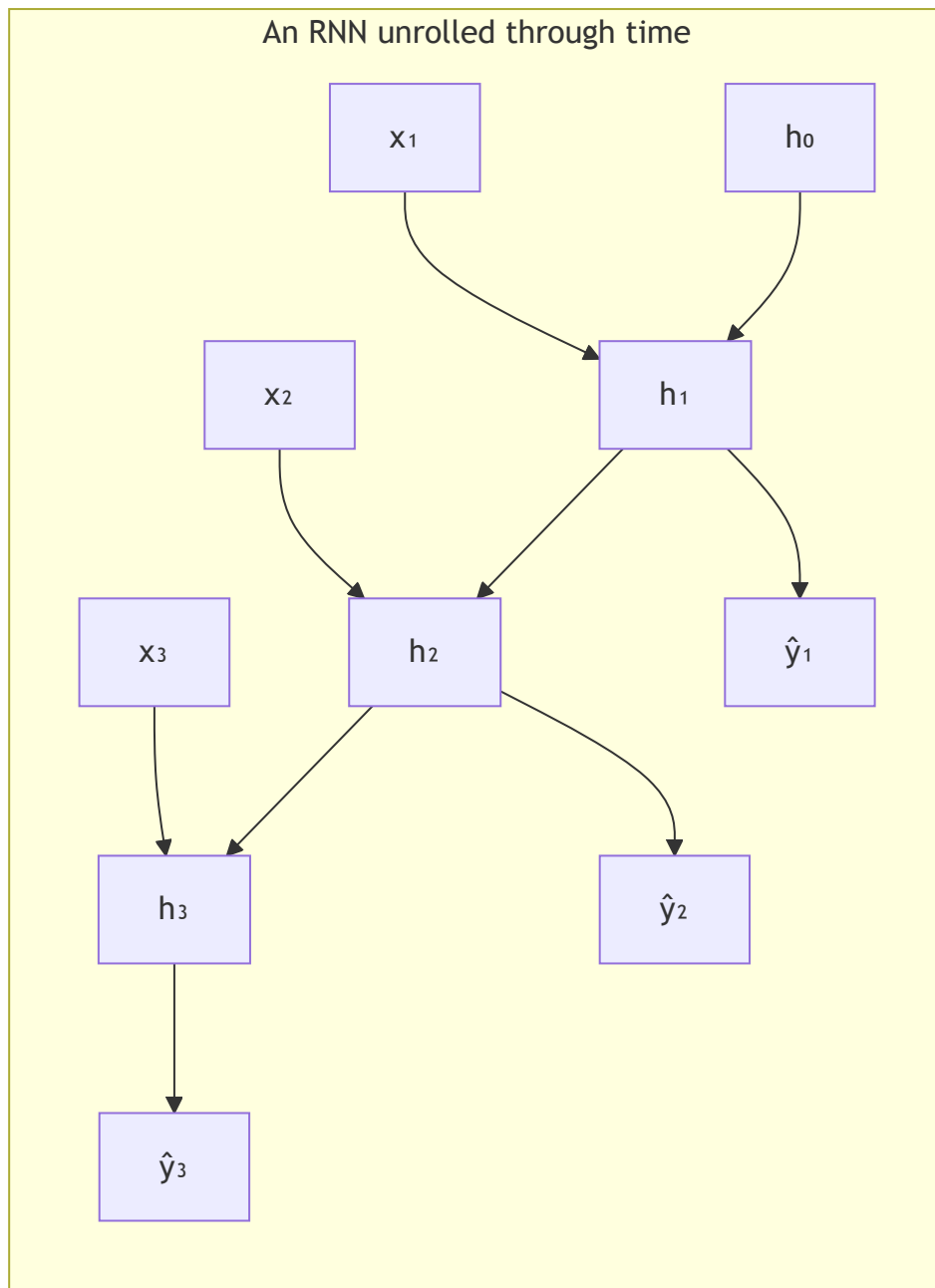
The feedforward NNLM from Chapter 0 could only look at a **fixed window** of previous words. It cannot use information from far back, and it treats a sentence as a bag of a few neighbours rather than an ordered stream.

The RNN fixes this by design:

- **Arbitrary-length context** — the hidden state can, in principle, carry information from the very first word to the last.
 - **Order matters** — the state is updated sequentially, so word order is naturally encoded.
 - **Parameter sharing** — one set of weights is reused at every time step, so the model size does not grow with sequence length.
-

1.3 Complete architecture

The recurrence (folded and unrolled)



The "folded" view is a single cell with an arrow looping back to itself. The "unrolled" view above shows the same cell copied once per time step — this is how you actually think about training it.

Components

Component	Symbol	Role
Input vector	x_t	Embedding of the current token.
Hidden state	h_t	The memory; a vector summarizing everything up to step t .
Input→hidden weights	W_{xh}	How the new input influences the state.
Hidden→hidden weights	W_{hh}	How the previous memory carries forward (the recurrent weights).

Component	Symbol	Role
Hidden→output weights	W_{hy}	How the state produces a prediction.
Biases	b_h, b_y	Learnable offsets.
Activation	$\tanh$	Non-linearity that keeps state values bounded in $[-1, 1]$.

1.4 How it does language modelling (step by step)

At every time step t :

1. Update the hidden state from the previous state and current input:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

2. Produce output logits from the new state:

$$z_t = W_{hy}h_t + b_y$$

3. Convert to a probability distribution over the vocabulary:

$$\hat{y}_t = \text{softmax}(z_t)$$

For language modelling, $\hat{y}_t$ is $P(w_{t+1} \mid w_1, \dots, w_t)$ — the predicted next word given all previous words. To **generate** text you sample a word from $\hat{y}_t$, feed it back in as x_{t+1} , and repeat.

Concrete pass for "the cat sat":

1. $x_1 = \text{embedding}(\text{"the"})$, $h_1 = \tanh(W_{xh}x_1 + W_{hh}h_0 + b_h) \rightarrow \text{predict "cat"}$.
2. $x_2 = \text{embedding}(\text{"cat"})$, h_2 folds "the" *and* "cat" into memory $\rightarrow \text{predict "sat"}$.
3. $x_3 = \text{embedding}(\text{"sat"})$, h_3 now summarizes the whole phrase $\rightarrow \text{predict next word}$.

1.5 Training — Backpropagation Through Time (BPTT)

The loss is the **cross-entropy** between the predicted distribution and the true next word, summed over all time steps:

$$L = - \sum_{t=1}^T \log \hat{y}_t[w_{t+1}]$$

Why this loss makes sense (intuition)

Read the term $\hat{y}_t[w_{t+1}]$ first: it is the probability the model assigned to the **word that actually came next**. A perfect model would give that word probability 1; a clueless model would give it something tiny. So this single number is a report card for the prediction at step t — "how much confidence did you put on the truth?"

Now walk through the pieces of the formula and *why each one is there*:

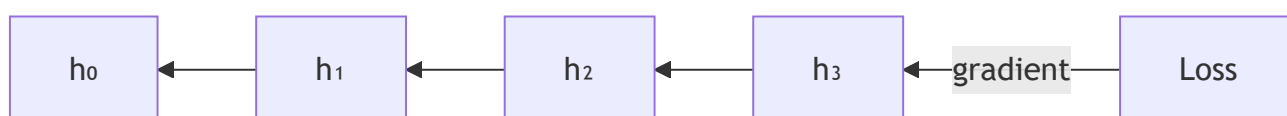
- **Why the log?** We want a penalty that is gentle when the model is right and brutal when it is confidently wrong. $-\log(p)$ does exactly that: if the model gives the true word $p = 1$, the penalty is $-\log(1) = 0$ (no complaint). As $p \rightarrow 0$, the penalty $-\log(p) \rightarrow \infty$ (a huge complaint). Being confidently wrong is punished far more than being merely unsure.

Prob. given to true word p	Loss $-\log p$	Meaning
1.0	0	perfect, no penalty
0.5	0.69	unsure
0.1	2.30	wrong and fairly confident
0.01	4.60	wrong and very confident → big push to fix

- **Why the minus sign?** Probabilities are between 0 and 1, so their logs are **negative**. The minus flips the loss to be **positive**, so "lower loss = better" holds and gradient *descent* pushes in the right direction.
- **Why only the true word's probability, and not the others?** Because the softmax already ties all the probabilities together — they must sum to 1. Pushing the true word's probability **up** automatically pulls the wrong words' probabilities **down**. So scoring just the correct word is enough to shape the whole distribution.
- **Why sum over all time steps t ?** Every position in the sequence is its own next-word prediction ("the" → "cat", "cat" → "sat", ...). Summing means the model is graded on **every** prediction it makes, so a single sentence gives T separate learning signals instead of one.

In one sentence: the loss rewards the model for putting high probability on the word that actually occurs, at every step, and grows sharply the more confidently it backs the wrong word — which is exactly the behaviour we want a language model to learn.

To train, we **unroll** the network across the whole sequence and apply ordinary backpropagation to the unrolled graph. Because the same weights appear at every step, the gradient for each weight is the **sum of its contributions across all time steps**. This algorithm is called **Backpropagation Through Time (BPTT)**.



The gradient at an early step must flow **backward through every intermediate step** — and this is exactly where the fatal problem appears.

How the loss propagates back into the weights

Backprop is just the chain rule applied along the unrolled graph. Follow the arrows in the recurrence backwards, from the loss to each weight matrix.

Step 1 — from loss to the output logits. For the prediction at step t , the gradient of cross-entropy through the softmax has a famously clean form:

$$\frac{\partial L}{\partial z_t} = \hat{y}_t - y_t$$

i.e. **predicted distribution minus the one-hot truth**. Intuitively: the probability mass you put on wrong words is pushed **down** and the mass on the true word is pushed **up**. The size of the error signal is literally "how wrong the distribution was."

Step 2 — the output weights W_{hy} . These sit between the hidden state and the logits, so their gradient is just the local error times the state that produced it, summed over every step:

$$\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^T (\hat{y}_t - y_t) h_t^\top$$

Step 3 — from logits back into the hidden state. Each state h_t affects the loss in **two** ways, and both paths must be added:

1. **Directly**, through its own prediction z_t .
2. **Indirectly**, through the next state h_{t+1} (because h_{t+1} was computed from h_t), which in turn affects *its* prediction and everything after it.

$$\frac{\partial L}{\partial h_t} = \underbrace{W_{hy}^\top (\hat{y}_t - y_t)}_{\text{from this step's prediction}} + \underbrace{W_{hh}^\top (\text{grad from } h_{t+1}) \odot \tanh'(\cdot)}_{\text{carried back from the future}}$$

This is the recursion: the error at step t is its own local error **plus** the error handed back from step $t + 1$. Computing it from the last step to the first is what "through time" means.

Step 4 — the recurrent and input weights W_{hh} , W_{xh} . Since W_{hh} and W_{xh} are reused at **every** step, each step contributes a piece and we **sum** them all:

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \delta_t h_{t-1}^\top, \quad \frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^T \delta_t x_t^\top$$

where $\delta_t = \frac{\partial L}{\partial h_t} \odot \tanh'(\cdot)$ is the error at the state *before* the non-linearity. This summing is the concrete meaning of "**the gradient for a shared weight is the sum of its contributions across all time steps.**"

Step 5 — update. Once every gradient is accumulated, the weights take one step downhill:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

with learning rate η . The same shared matrices then produce better predictions on the next pass.

The key thing to notice is the term **carried back from the future** in Step 3: to reach an early state h_1 , the error must pass through W_{hh} and $\tanh'$ once for **every** step in between — and that repeated multiplication is exactly what breaks down next.

1.6 Limitations — vanishing and exploding gradients

When the gradient flows back through many steps, it gets **repeatedly multiplied** by the recurrent weight matrix W_{hh} and by the derivative of $\tanh$. Consider the chain that connects a far-apart pair of steps:

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}}$$

This is a **product of many terms**:

- If those terms are < 1 on average, the product shrinks exponentially → the gradient **vanishes**. Early words contribute almost nothing to learning. The RNN effectively has a **short memory** (a few steps).
- If those terms are > 1 on average, the product grows exponentially → the gradient **explodes**, and training becomes unstable (fixable with gradient clipping, but the vanishing case is not so easily patched).

The practical consequences

Limitation	Effect on language modelling
Vanishing gradients	Cannot learn long-range dependencies (e.g. subject–verb agreement across a long clause).
Short effective memory	In practice remembers only ~5–10 steps well.
Strictly sequential	Step t needs step $t - 1$, so it cannot be parallelized across time — training is slow.
Single state bottleneck	All history is crushed into one fixed-size vector.

1.7 How it gave rise to the next model

The RNN got the *shape* of the solution right — a recurrent memory — but the vanishing gradient meant the memory was too short to be useful for real language.

The fix is not to remove recurrence, but to redesign the **cell** so that information can flow across many steps **without being repeatedly squashed**. The key insight: add a protected memory channel and use learnable **gates** to control what is remembered, forgotten, and output. That redesign is the **Long Short-Term Memory (LSTM)** cell.

 Continue to [Chapter 2 — LSTM & GRU](#)

Chapter 2 — LSTM & GRU

2.1 What it is

The **Long Short-Term Memory (LSTM)** network is an RNN with a redesigned cell. It keeps the recurrence idea but adds two things:

1. A **cell state** C_t — a protected "conveyor belt" of memory that runs straight through time with only minor, controlled edits.
2. **Gates** — small neural networks that decide, at each step, what to **forget**, what to **add**, and what to **output**.

The **GRU (Gated Recurrent Unit)** is a lighter, later simplification of the same idea.

The one-line intuition: instead of overwriting memory at every step (which is what causes vanishing gradients), the LSTM *chooses* how much of the old memory to keep and how much new information to write.

2.2 Why it appeared (the limitation it fixed)

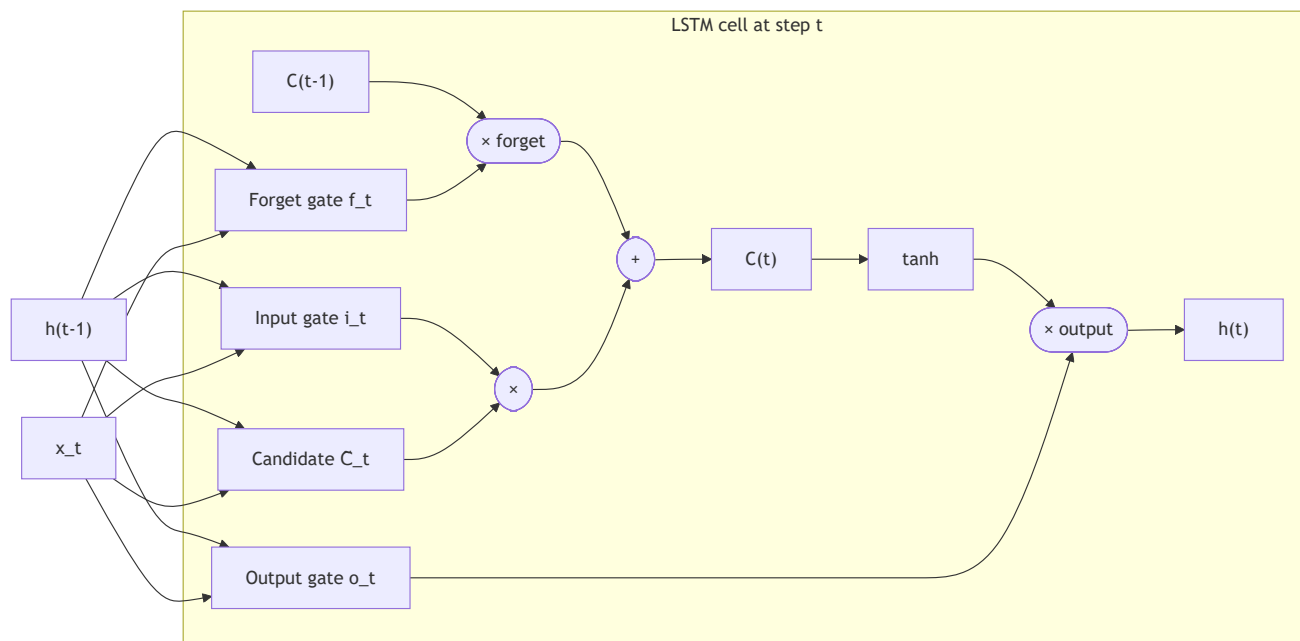
Recall the RNN's fatal flaw: repeatedly multiplying gradients through $\tanh$ and W_{hh} makes them vanish, so the network cannot learn dependencies more than a handful of steps apart.

The LSTM fixes this with the **cell state and its additive update**. Instead of transforming the whole memory every step (multiplicative, gradient-destroying), the cell state is updated by **adding** information:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Because the path from C_{t-1} to C_t is mostly **addition** gated by f_t , gradients can flow backward across many steps **without vanishing** — this protected path is often called the **constant error carousel**. This is the single most important idea in the chapter.

2.3 Complete architecture



The three gates and the candidate

Each gate looks at the previous hidden state h_{t-1} and the current input x_t , and outputs values between 0 and 1 (via a **sigmoid** σ) that act as "how much to let through" dials.

Component	Formula	Role in plain language
Forget gate	$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$	How much of the old cell memory to keep (1 = keep all, 0 = erase).
Input gate	$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$	How much of the new candidate to write.
Candidate	$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$	The new information proposed for storage.
Output gate	$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$	How much of the cell state to expose as the hidden state.

Here $[h_{t-1}, x_t]$ means the two vectors concatenated.

2.4 How it works — the update, step by step

1. Decide what to forget. The forget gate reads the context and outputs a keep/erase value for each dimension of memory:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

2. Decide what new information to store. The input gate chooses *how much*, the candidate proposes *what*:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad \tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

3. Update the cell state — forget some old, add some new (the crucial additive step):

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

4. Produce the hidden state / output — a filtered view of the cell state:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(C_t)$$

$\odot$ is element-wise multiplication. The hidden state h_t is what gets passed to the next step and used (through a softmax layer, just like an RNN) to predict the next word.

Language-modelling example: in "The clouds are in the __", the cell state can carry "clouds/sky context" across the intervening words because the forget gate keeps it and the input gate does not overwrite it — so the model correctly predicts "sky". A vanilla RNN would likely have lost that signal.

2.5 The GRU — a simpler alternative

The **Gated Recurrent Unit** merges ideas to use **two gates instead of three** and **no separate cell state** (the hidden state carries memory directly). It is cheaper and often performs comparably.

Component	Formula	Role
Update gate	$z_t = \sigma(W_z[h_{t-1}, x_t])$	Balances keeping old state vs. taking new (combines LSTM's forget + input).
Reset gate	$r_t = \sigma(W_r[h_{t-1}, x_t])$	How much past state to use when forming the candidate.
Candidate	$\tilde{h}_t = \tanh(W[h_t \odot r_t, h_{t-1}, x_t])$	Proposed new state.
Final state	$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$	Interpolate between old and new.

LSTM vs GRU: LSTM has more capacity and an explicit memory cell (often better on very long or complex sequences); GRU has fewer parameters and trains faster. Both solve the vanishing-gradient problem the same way — with gated, additive state updates.

2.6 Limitations

LSTMs and GRUs powered most of NLP from roughly 2014–2017, but they still carry deep limitations:

Limitation	Why it hurts
Still sequential	Step t depends on step $t - 1$. Cannot parallelize across the time axis → slow training on modern hardware.
Very long dependencies still fade	Gates <i>mitigate</i> but do not fully eliminate signal loss over hundreds of steps.
Single fixed-size state	The entire past is still compressed into one vector — a bottleneck for long or information-dense sequences.
No direct access to the past	To use word 3 at step 100, the info must survive 97 sequential updates; there is no way to "jump back" and read it directly.

That last point becomes critical the moment we ask an RNN/LSTM to do a **sequence-to- sequence** task like translation.

2.7 How it gave rise to the next model

LSTMs made recurrent memory *usable*. The next question was structural, not about the cell: **how do you map one whole sequence to a different sequence** — English to French, question to answer, article to summary — when the input and output can have different lengths?

The answer was to connect two LSTMs: one to **read** (encode) the input into a summary vector, and one to **write** (decode) the output from it. That is the **Sequence-to- Sequence (Seq2Seq)** architecture — and its central weakness (the single summary vector) is what directly motivates attention.

Why LSTMs struggle at sequence-to-sequence

When you force an LSTM to turn one whole sequence into another (e.g. translate a sentence), these problems show up in plain terms:

- **One tiny vector must hold the whole input.** The encoder squeezes the entire sentence into a single fixed-size vector. Long sentences don't fit — early words get "squeezed out" before the decoder ever starts writing.
- **Beginning of the sentence gets forgotten.** By the time the encoder reaches the last word, the first few words have faded, so translations of long inputs drift or lose detail.
- **The decoder can't look back at specific words.** It only sees the one summary vector — it cannot "re-read" the exact input word it needs while producing each output word.
- **Fixed-size memory, variable-size meaning.** A 5-word and a 50-word sentence are both crammed into the same-size vector, so more content means more information loss.
- **Errors pile up (exposure bias).** The decoder generates one word at a time feeding on its own previous outputs; one early mistake can cascade into the rest of the sentence.

- **No alignment between input and output.** Translation needs to know *which* source word maps to *which* target word; a single summary vector throws that word-to-word alignment away.
- **Still strictly sequential.** Both reading and writing happen step-by-step, so it is slow to train and can't exploit parallel hardware.
- **A plain encoder can't look forward (softer, solvable).** A unidirectional LSTM encodes word t having only seen the words before it, so it misses future context (e.g. "bank" in "river bank" vs "bank account"). This one *can* be fixed within LSTMs by using a **Bidirectional LSTM (BiLSTM)** — one pass left→right, one right→left, concatenated — so unlike the points above, it isn't what forced the move to attention.

Every one of these points is exactly what **attention** was invented to fix — by letting the decoder look back at *all* encoder states and focus on the relevant ones for each output word.

 Continue to [Chapter 3 — Seq2Seq](#)

Chapter 3 — Sequence-to-Sequence (Seq2Seq)

3.1 What it is

Seq2Seq (Sutskever et al., 2014) is an architecture for mapping an **input sequence** to an **output sequence** of possibly different length. It joins two recurrent networks:

- An **Encoder** reads the entire input sequence and compresses it into a single fixed-size vector called the **context vector** (or "thought vector").
- A **Decoder** takes that context vector and generates the output sequence one token at a time.

This is the architecture that first made neural machine translation, summarization, and conversational response generation practical.

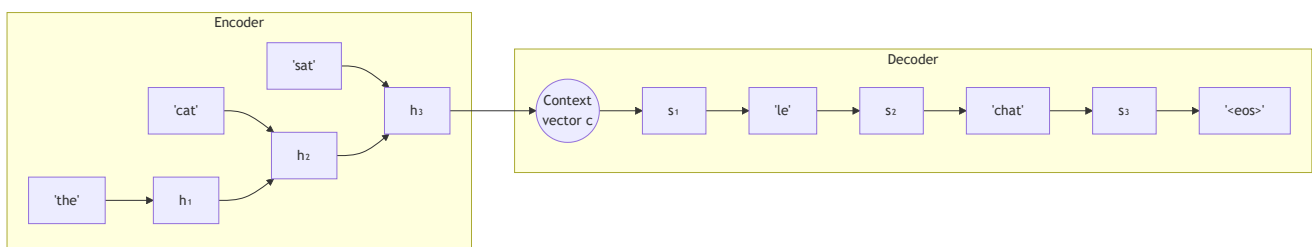
3.2 Why it appeared (the limitation it fixed)

A plain RNN/LSTM produces **one output per input step** — the input and output are locked to the same length and the same positions. But many of the most important language tasks are **many-to-many with different lengths**:

- Translation: 5 English words → 8 French words.
- Summarization: 500-word article → 30-word summary.
- Dialogue: a question → an answer of unrelated length.

A single LSTM cannot do this. Seq2Seq solves it by **separating reading from writing**: first fully read the input (encoder), *then* start writing the output (decoder). The context vector is the handoff between the two.

3.3 Complete architecture



Components

Component	Role
Encoder RNN/LSTM	Reads input tokens $x_1 \dots x_T$, updating its hidden state each step.
Context vector c	The encoder's final hidden state h_T — a fixed-size summary of the whole input.
Decoder RNN/LSTM	Initialized with c ; generates output tokens autoregressively.
<sos> / <eos> tokens	Special markers telling the decoder when to start and stop.
Softmax output layer	Turns each decoder state into a probability distribution over the target vocabulary.

3.4 How it works — step by step

Encoding

The encoder runs like a normal LSTM over the input, but we **only keep the final state**:

$$h_t = \text{LSTM}(h_{t-1}, x_t), \quad c = h_T$$

All the meaning of the input sentence is now packed into the single vector c .

Decoding

The decoder is itself a language model, but **conditioned on** c . Its first state is set from c , and it generates one word at a time, feeding each generated word back in as the next input:

$$s_t = \text{LSTM}(s_{t-1}, y_{t-1}), \quad P(y_t | y_{<t}, c) = \text{softmax}(W s_t + b)$$

It starts from the **<sos>** token and continues until it emits **<eos>**.

Two decoding-time details

- **Teacher forcing (training)**: during training the decoder is fed the *true* previous target word (not its own prediction), which stabilizes and speeds up learning.
- **Beam search (inference)**: instead of greedily taking the single most likely word at each step, keep the top- k partial sequences and expand them, which finds higher- probability overall outputs.

Training objective

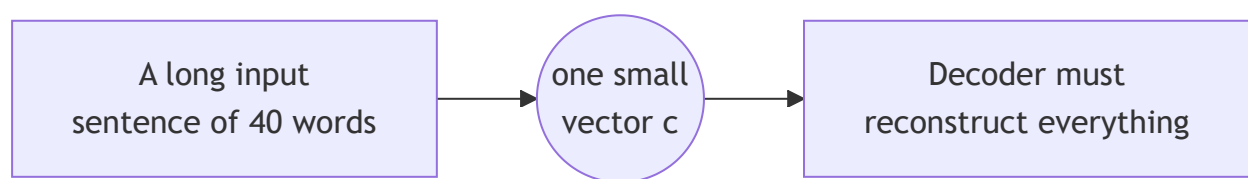
Maximize the probability of the correct output sequence (equivalently, minimize cross-entropy) over the whole target:

$$L = - \sum_{t=1}^{T'} \log P(y_t | y_{<t}, c)$$

3.5 Limitations — the fixed-vector bottleneck

Seq2Seq has one dominant, well-known weakness that defines the next chapter:

Everything about the input — no matter how long — must be crammed into a single fixed-size context vector c .



Limitation	Consequence
Information bottleneck	A fixed vector cannot hold all details of a long sentence; quality drops sharply as input length grows.
Recency bias	The final state h_T remembers the <i>end</i> of the input better than the beginning.
No alignment	When generating the French word for "cat", the decoder cannot point back to <i>where</i> "cat" was in the input — it only has the blurred summary.
Still sequential	Inherits the RNN's inability to parallelize across time.

The empirical signature: translation BLEU scores fell off a cliff for long sentences.

What is a BLEU score? BLEU (Bilingual Evaluation Understudy) is the standard automatic metric for machine translation. It measures how much the machine's output overlaps with one or more human reference translations by counting matching word sequences (n-grams), then applies a penalty for outputs that are too short. It ranges from **0 to 100** — higher is better, with human-quality translations typically scoring in the 30–50 range. "Fell off a cliff" means the score dropped sharply as sentences got longer, confirming the bottleneck.

3.6 How it gave rise to the next model

The problem is the **single** context vector. The obvious fix: instead of forcing the decoder to rely on one summary, let it **look back at all the encoder's hidden states** $h_1, \dots, h_T$ and, at each output step,

focus on the most relevant ones.

When generating "chat", the decoder should be able to pay most of its attention to the encoder state for "cat". This mechanism — a learned, weighted look-back over the entire input — is called **Attention**, and it removes the bottleneck entirely.

 Continue to [Chapter 4 — Attention](#)

Chapter 4 — Attention

4.1 What it is

Attention is a mechanism that lets a model, at each output step, **look back over all the input representations and take a weighted average of them**, where the weights say "how relevant is each input position to what I am producing right now."

Instead of the decoder relying on a single frozen context vector, attention builds a **fresh, custom context vector for every output token**, assembled on demand from the parts of the input that matter most at that moment.

First introduced by Bahdanau et al. (2014) for translation, attention is the conceptual bridge to the Transformer.

4.2 Why it appeared (the limitation it fixed)

Seq2Seq forced the entire input through one fixed-size bottleneck vector c . Long inputs overflowed it, and the decoder had no way to align its output with specific input words.

Attention removes the bottleneck directly:

- **No single summary** — the decoder keeps access to *all* encoder states $h_1, \dots, h_T$.
 - **Dynamic focus** — for each output word it computes new weights, focusing on the relevant input words.
 - **Alignment for free** — the attention weights literally show which input word each output word is "looking at" (interpretable alignment).
-

4.3 The core mechanism — Query, Key, Value

Attention is best understood through three roles. Think of it as a **soft dictionary lookup**:

Role	Meaning	In Seq2Seq attention
Query (q)	What I am looking for right now.	The current decoder state s_{t-1} .
Key (k)	A label for each stored item, used for matching.	Each encoder state h_i .
Value (v)	The actual content returned when a key matches.	Each encoder state h_i .

The process every output step:

1. Score — compare the query against every key to get a relevance score:

$$e_{ti} = \text{score}(s_{t-1}, h_i)$$

2. Normalize — turn scores into weights that sum to 1 with a softmax:

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_{j=1}^T \exp(e_{tj})}$$

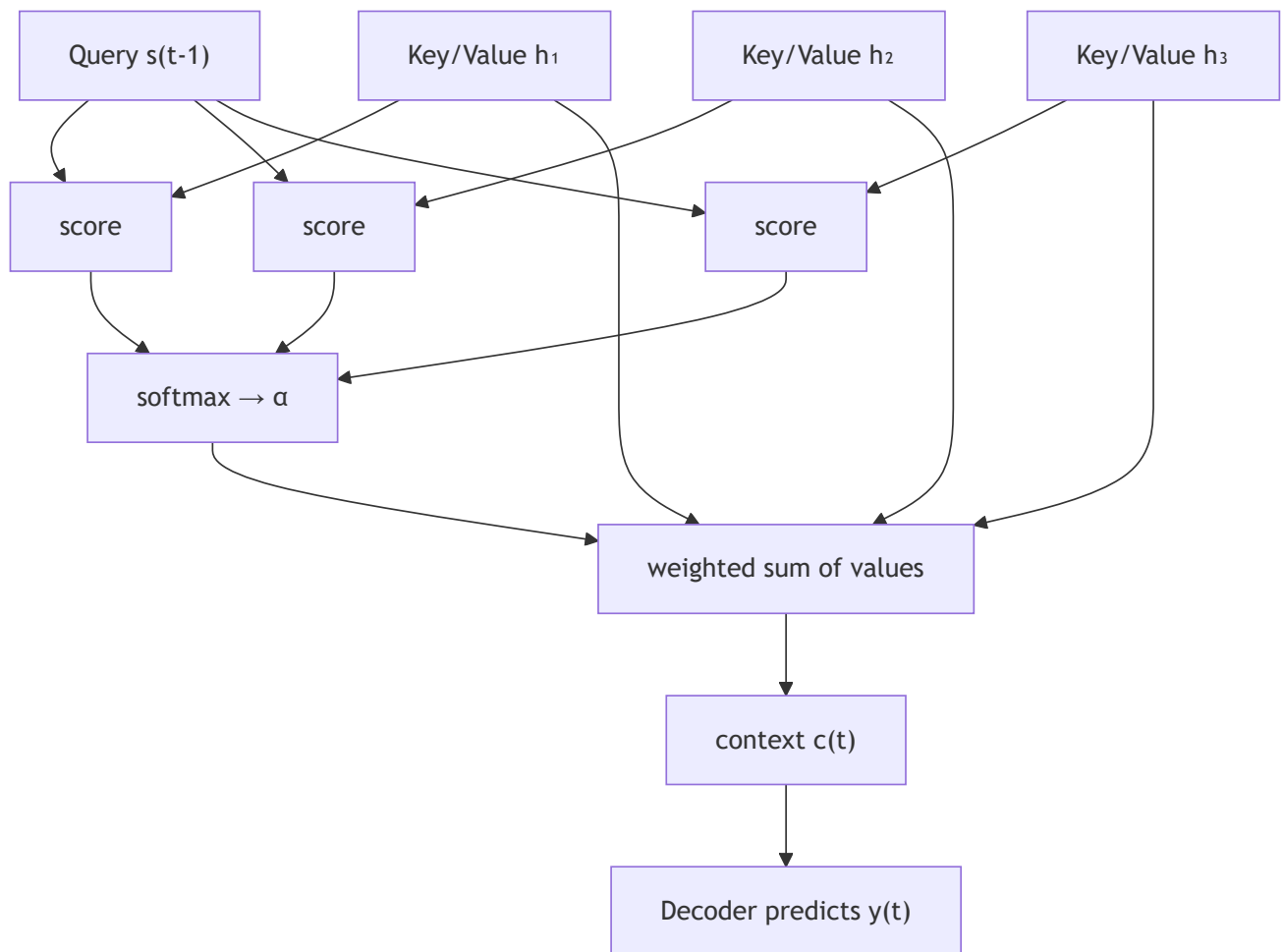
3. Aggregate — take the weighted sum of the values to form the context vector:

$$c_t = \sum_{i=1}^T \alpha_{ti} h_i$$

What the symbols mean:

Symbol	Meaning
t	The current output (decoder) step — the word being generated right now.
i	An index over input positions (1 ... T) — which encoder word we are looking at.
T	The number of words in the input sequence.
s_{t-1}	The decoder's hidden state from the previous step — the query ("what I need now").
h_i	The encoder's hidden state for input word i — used as both key and value .
$\text{score}(\cdot)$	A function measuring how relevant input word i is to the current step (e.g. dot product).
e_{ti}	The raw relevance score between output step t and input word i .
α_{ti}	The normalized attention weight (0–1, all summing to 1) — how much focus word i gets at step t .
$\exp(\cdot)$	The exponential function, used by softmax to turn scores into positive weights.
c_t	The context vector for step t : a weighted blend of encoder states, emphasizing the relevant ones.

This c_t is now **specific to output step t** , and is fed to the decoder together with its state to predict the next word.



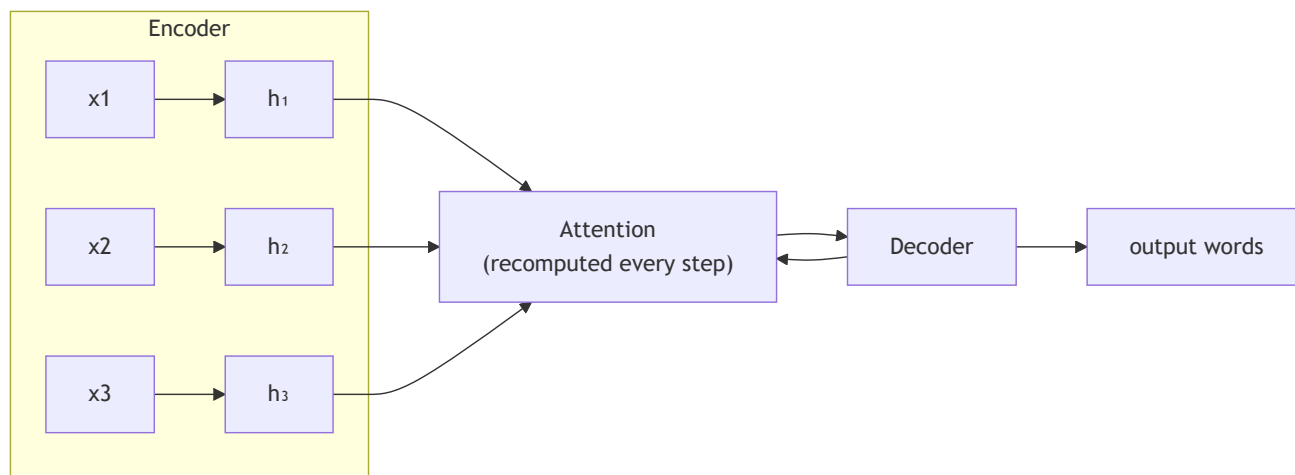
4.4 Scoring functions

The **score** function measures query–key similarity. The two classic forms:

Name	Formula	Notes
Additive / Bahdanau	$\text{score}(s, h) = v^T \tanh(W_1 s + W_2 h)$	A small neural network; the original 2014 form.
Multiplicative / Luong (dot)	$\text{score}(s, h) = s^T h$	Cheaper; just a dot product. Basis of the Transformer.

The dot-product form is important because it is fast and parallelizable — a property the Transformer exploits fully.

4.5 How attention changes Seq2Seq



Compared to plain Seq2Seq, the only change is that the decoder, at each step, computes c_t by attending over **all** encoder states rather than reusing a single c . The encoder and decoder are still LSTMs.

The alignment weights α_{ij} can be plotted as a heatmap and reveal sensible word-to-word correspondences (e.g. an English word lighting up under its French translation) — the first time neural translation became *interpretable*.

4.6 Self-attention — the pivotal generalization

Bahdanau attention connected a **decoder to an encoder**. The decisive next idea: apply attention of a sequence **to itself**.

In **self-attention**, the queries, keys, and values all come from the *same* sequence. Every word attends to every other word in the same sentence, directly modelling relations like "which noun does this pronoun refer to" — regardless of distance.

each word x_i builds q_i, k_i, v_i from itself, then attends over all x_j

This is transformative because:

- It connects any two positions in **one step** (no long chain of recurrence) → **no vanishing gradient over distance**.
- All positions are computed **simultaneously** → **fully parallelizable**.

At this point a natural question arises: *if attention alone can relate any two positions directly and in parallel, do we even need recurrence anymore?*

4.7 Limitations of attention *added onto RNNs*

Limitation	Consequence
Recurrence still present	The encoder/decoder are still LSTMs, so training is still sequential and slow .
Attention is a bolt-on	It fixes the bottleneck but sits on top of an architecture that fundamentally cannot parallelize.
Complexity	Two mechanisms (recurrence + attention) to build, tune, and reason about.

4.8 How it gave rise to the next model

The 2017 paper title says it all: "**Attention Is All You Need.**" The insight was to **throw away recurrence entirely** and build the whole model out of (self-)attention plus simple feedforward layers. This removes the sequential bottleneck, enables massive parallel training on GPUs/TPUs, and connects distant words directly.

That architecture is the **Transformer** — the foundation of every model that follows.

 Continue to [Chapter 5 — Transformer](#)

Chapter 5 — The Transformer

The single most important architecture in modern NLP. Introduced in "*Attention Is All You Need*" (Vaswani et al., 2017). Every model after this chapter is a Transformer variant.

5.1 What it is

The **Transformer** is a sequence model built **entirely from attention and feedforward layers** — **with no recurrence and no convolution**. It processes all tokens of a sequence **in parallel**, and uses **self-attention** so that every token can directly attend to every other token.

It keeps the encoder–decoder shape of Seq2Seq but replaces the LSTMs with stacks of attention-based blocks.

5.2 Why it appeared (the limitation it fixed)

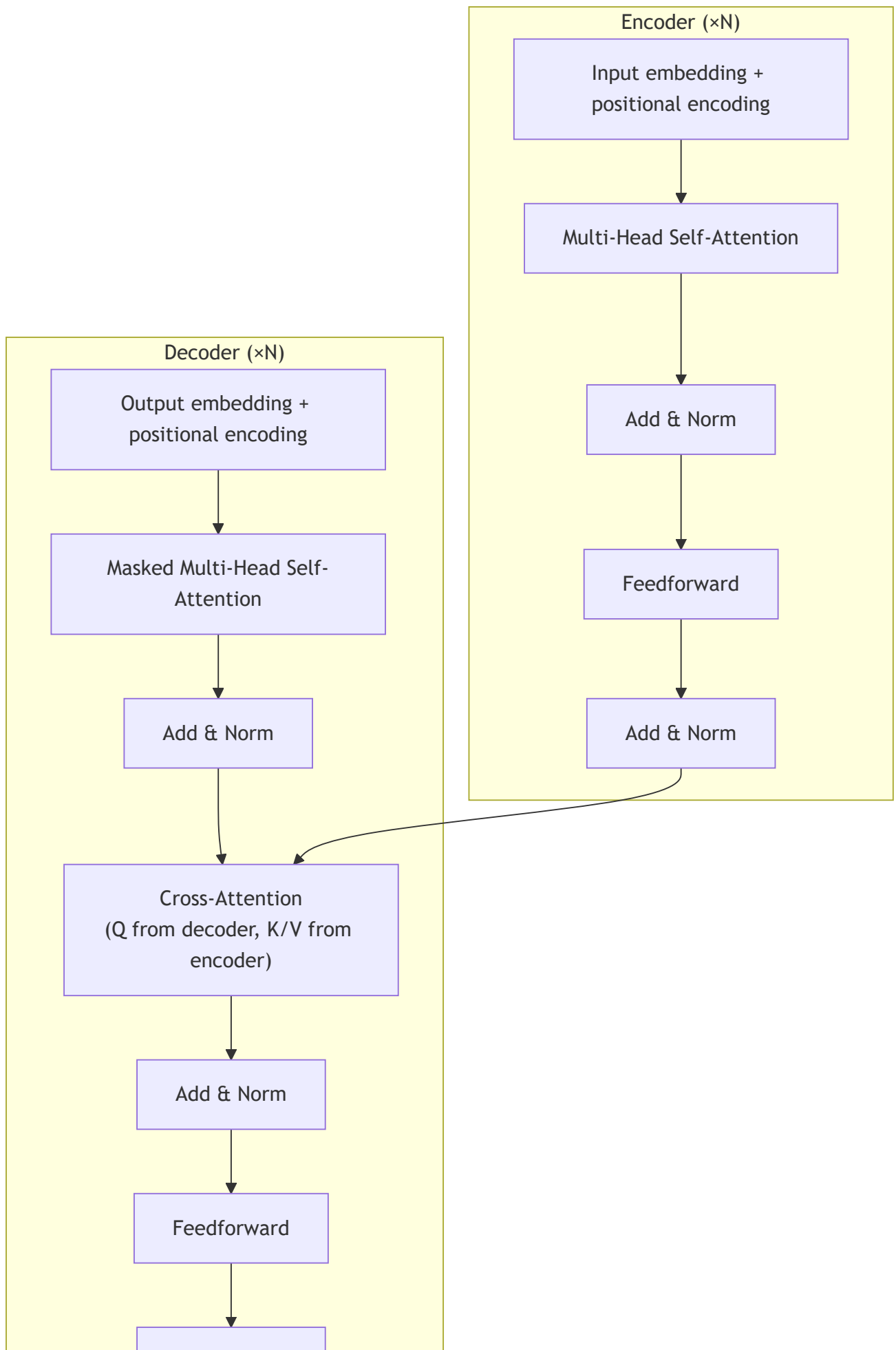
RNN/LSTM-based models — even with attention bolted on — had two unavoidable problems:

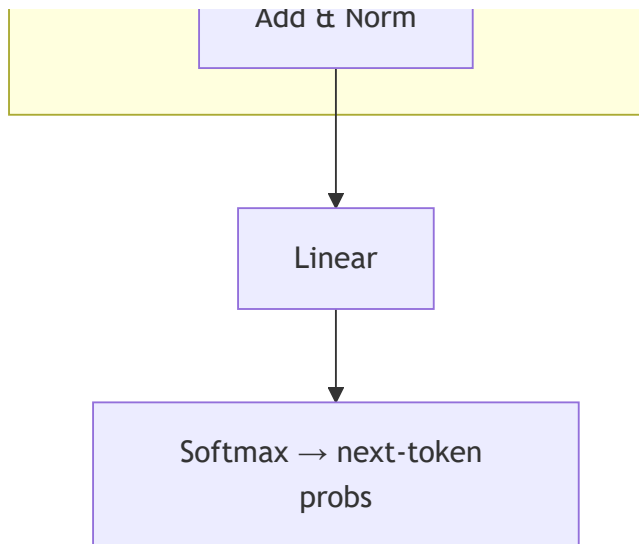
1. **They are sequential.** Step t needs step $t - 1$, so training cannot be parallelized across the sequence, wasting modern hardware.
2. **Distant tokens are hard to relate.** Information must survive many recurrent steps.

The Transformer fixes both at once:

Problem	Transformer's fix
Sequential, slow	Self-attention processes all positions simultaneously → full GPU/TPU parallelism.
Long-range dependencies	Any two tokens are connected in one attention step , path length $O(1)$.
Fixed bottleneck	Every token keeps its own representation; no single summary vector.

5.3 Complete architecture





The original Transformer stacks $N = 6$ identical encoder blocks and $N = 6$ decoder blocks. Let us build up every component.

5.4 Component 1 — Input embeddings + Positional Encoding

Since there is **no recurrence**, the model has no inherent sense of word order — feeding tokens in parallel loses position information. The fix is to **add a positional signal** to each token embedding.

The original paper uses fixed sinusoids:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- pos = position in the sequence, i = dimension index, d = embedding size.
- Each position gets a unique pattern; the sinusoidal form lets the model generalize to relative positions and to lengths unseen in training.

The final input to the first layer is $\text{embedding}(w) + PE(pos)$. (Modern models often use learned or rotary positional encodings instead, but the purpose is identical.)

5.5 Component 2 — Scaled Dot-Product Attention

This is the engine. For each token we form a **query**, **key**, and **value** by multiplying its embedding by learned matrices W^Q, W^K, W^V . Stacking all tokens into matrices Q, K, V :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Reading the formula piece by piece:

Piece	Meaning
$QK^\top$	Every query dotted with every key $\rightarrow$ an $n \times n$ matrix of raw relevance scores.
$\div \sqrt{d_k}$	Scaling. Without it, large dot products push softmax into tiny-gradient regions; dividing by $\sqrt{d_k}$ keeps values stable.
softmax	Turns each row of scores into weights that sum to 1.
$\times V$	Weighted sum of value vectors $\rightarrow$ the new, context-aware representation of each token.

Because this is pure matrix multiplication, **all tokens are processed at once**.

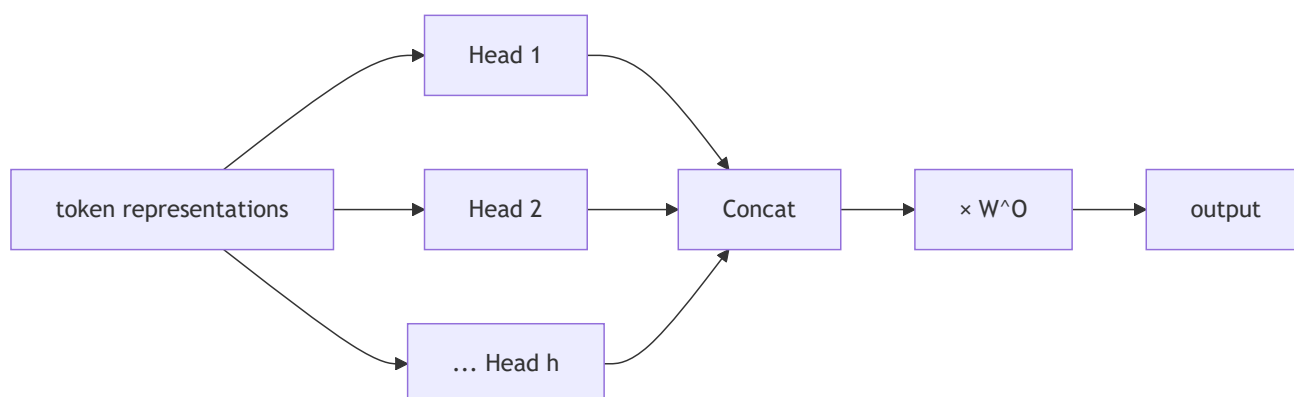
5.6 Component 3 — Multi-Head Attention

One attention "head" can only capture one type of relationship. **Multi-head attention** runs h attentions in parallel, each with its own W_i^Q, W_i^K, W_i^V , then concatenates and projects the results:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Intuition: different heads learn different relationships — one head might track subject–verb agreement, another might track pronoun references, another positional proximity. The original model uses $h = 8$ heads.



5.7 Component 4 — Position-wise Feedforward Network

After attention mixes information *across* tokens, each token is independently transformed by a small 2-layer MLP (the same weights applied at every position):

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

It expands to a larger inner dimension (e.g. $4\times$), applies a non-linearity, and projects back. This adds representational capacity and non-linear processing per token.

5.8 Component 5 — Residual connections + Layer Normalization

Every sub-layer (attention and FFN) is wrapped as:

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

- **Residual connection** ($x + \dots$): lets gradients flow straight through deep stacks, making it possible to train many layers (echoing why the LSTM's additive path worked).
- **Layer normalization**: stabilizes and speeds up training by normalizing each token's features.

5.9 Encoder vs. Decoder — and the three kinds of attention

The Transformer uses attention in **three distinct places**:

Location	Query from	Key/Value from	Masked?	Purpose
Encoder self-attention	encoder	encoder	No	Each input token sees the whole input (bidirectional context).
Decoder masked self-attention	decoder	decoder	Yes	Each output token sees only tokens before it (preserves autoregression).
Encoder-decoder cross-attention	decoder	encoder	No	Each output token attends over the full input (this replaces Seq2Seq attention).

Why the decoder mask matters

When generating text, the model must not "peek" at future words. The **causal mask** sets the attention scores for future positions to $-\infty$ before the softmax, so their weights become 0:

$$\text{mask}_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

This single design choice — mask or no mask — is exactly what later splits the family tree into **BERT** (no mask, bidirectional) and **GPT** (masked, autoregressive).

5.10 How the Transformer does language modelling — full data flow

Let's trace a full pass end-to-end using a translation example: input **"the cat sat"** → output **"le chat"**. The same machinery underlies every Transformer-based language model.

Step 1 — From text to vectors

1. **Tokenize.** The text is split into tokens (roughly sub-words). "the cat sat" might become `["the", "cat", "sat"]`.
2. **Embed.** Each token id is looked up in an embedding table, giving a vector (say 512 numbers) that captures its meaning.
3. **Add position.** Self-attention has *no inherent sense of order*, so a **positional encoding** is added to each embedding to stamp in "I am word 1, 2, 3...". Now each vector encodes both *what* the word is and *where* it sits.

Step 2 — The Encoder reads the whole input (all at once)

The encoder is a stack of **N identical blocks**. Each block does two things, each followed by an *add & norm* (a residual shortcut plus normalization that keeps training stable):

- **Multi-head self-attention** — every input word looks at *every other* input word (including itself) and pulls in relevant context. "cat" can attend to "the" and "sat" simultaneously. Multiple "heads" let it capture several kinds of relationships at once (grammar, meaning, position...).
- **Feed-forward network (FFN)** — a small per-word neural net that further transforms each vector independently.

Crucially, all input words flow through this **in parallel** (no step-by-step recurrence). After N blocks, the encoder outputs one **context-rich vector per input word** — a deep, bidirectional understanding of "the cat sat".

Step 3 — The Decoder writes the output (one word at a time)

The decoder is also a stack of **N blocks**, but each block has *three* sub-layers (each with add & norm):

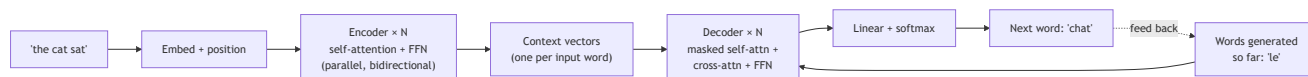
- **Masked self-attention** — the decoder attends to the words it has *already produced*, but a **mask** hides future positions so it can't "cheat" by seeing words it hasn't generated yet. When producing word 2, it may look at word 1 only.
- **Cross-attention** — this is the bridge to the encoder. The decoder's current state acts as the **query**, and the encoder's outputs are the **keys and values**. So while generating "chat", the decoder looks back at *all* the input words and focuses on "cat". This is the attention mechanism from Chapter 4, now built into the architecture.
- **Feed-forward network** — same per-word transformation as in the encoder.

Step 4 — Turn the top vector into a next-word prediction

The decoder's final vector for the current position goes through a **linear layer + softmax**, producing a probability distribution over the *entire vocabulary*. The highest-probability token is the model's prediction for the next word (e.g. "chat" after "le").

Step 5 — Training vs. generation

- **Training** minimizes **cross-entropy** against the correct next tokens. A trick called **teacher forcing** feeds the *real* previous word (not the model's guess) during training. Because the mask lets every position be scored simultaneously, **one forward pass scores an entire sentence at once** — this parallelism is what makes Transformers train so fast compared to RNNs.
- **Generation** (inference) is done one token at a time: predict a word, append it to the output, feed it back in, and repeat until an end-of-sequence token.



One-line summary: the **encoder** builds a deep understanding of the input in parallel, the **decoder** generates the output word-by-word while using **cross-attention** to look back at that understanding, and a **linear + softmax** turns each decoder state into the next word.

5.11 Limitations

Limitation	Explanation
Quadratic cost	Self-attention compares every token with every other → compute and memory scale as $O(n^2)$ with sequence length n . Long documents are expensive.
No built-in notion of order	Position must be injected artificially; getting this right (especially for long/extrapolated lengths) is an active research area.
Data and compute hungry	Its power is unlocked only with large datasets and large models.
Fixed context window	Can only attend within its trained window length.

Note these are limitations of *scale and cost*, not of *capability* — which is why the next step was not a new architecture but a new **training paradigm** on top of the Transformer.

5.12 How it gave rise to the next models

The Transformer is a general engine. The breakthrough realization of 2018 was about **how to train it**: rather than training from scratch on one task, **pre-train** a Transformer on enormous unlabeled text to learn language in general, then **fine-tune** it on specific tasks. Two complementary ways to do this pre-training emerged, split precisely along the masking choice from §5.9:

- Use only the **encoder** with bidirectional context, trained to fill in blanks → **BERT** (great for *understanding*).
- Use only the **decoder** with causal masking, trained to predict the next word → **GPT** (great for *generation*).

➔ Continue to [Chapter 6 — BERT](#) · then [Chapter 7 — GPT](#)

Chapter 6 — BERT

BERT = **B**idirectional **E**ncoder **R**epresentations from **T**ransformers (Devlin et al., 2018, Google).

6.1 What it is

BERT is a **stack of Transformer encoders** (the decoder is discarded) that is pre-trained on massive unlabeled text to produce **deeply bidirectional, contextual** representations of words. After pre-training once, it is **fine-tuned** with a small task-specific head for many downstream tasks (classification, question answering, named-entity recognition, etc.).

BERT is an **understanding** model, not a text generator. Its job is to read text and build rich representations of it.

Two sizes from the original paper:

Model	Layers	Hidden size	Heads	Parameters
BERT-Base	12	768	12	110M
BERT-Large	24	1024	16	340M

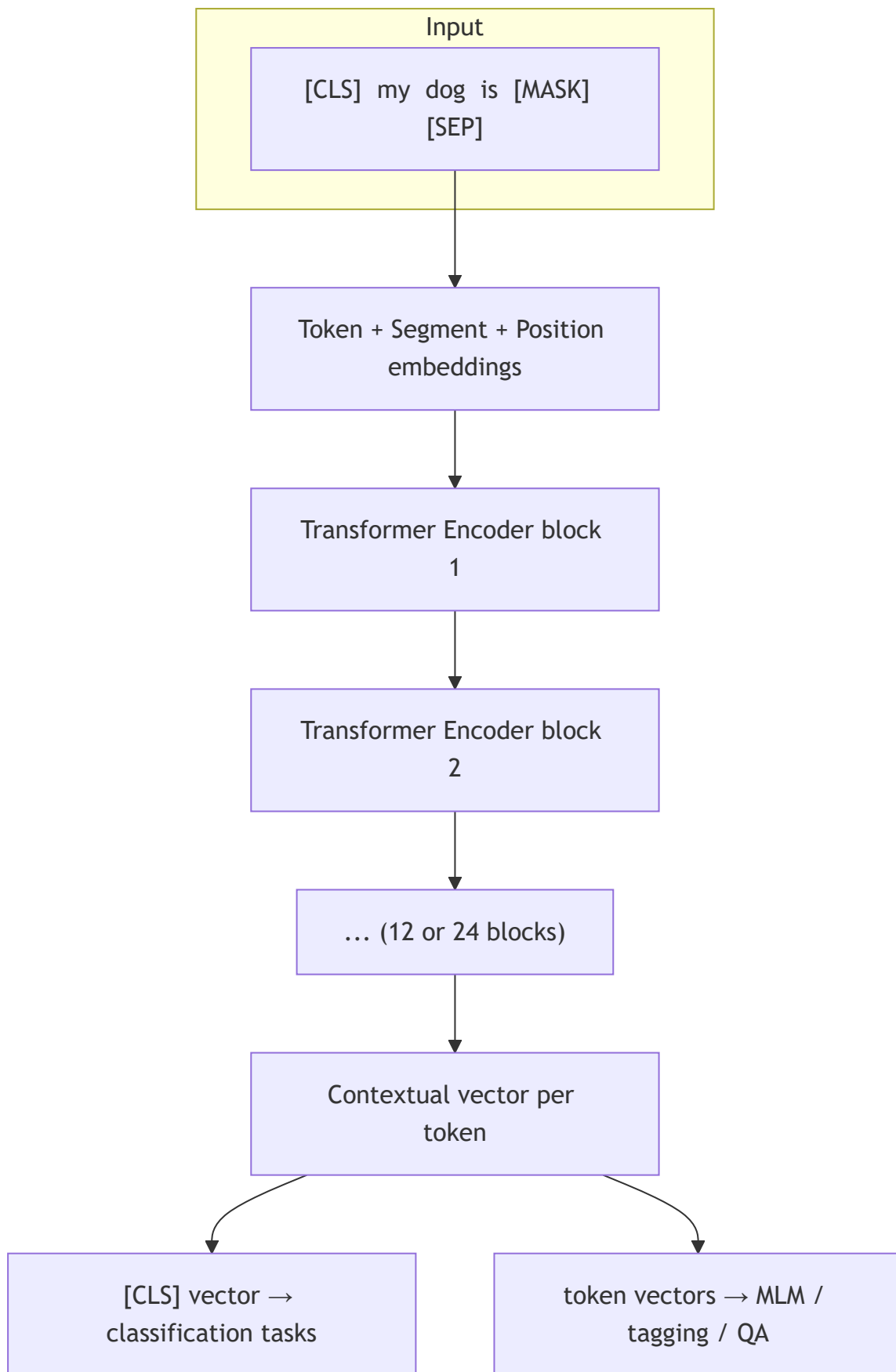
6.2 Why it appeared (the limitation it fixed)

Before BERT, contextual models were **unidirectional** — they read left-to-right (or ran two separate left-to-right and right-to-left models and concatenated them, as in ELMo). A left-to-right model representing "bank" in "the river bank" has not yet seen "river" if it reads forward, or misses later context.

But true understanding needs **both directions at once**. In "He went to the **bank** to withdraw money," the word "withdraw/money" *after* "bank" is what disambiguates it.

BERT's key move: use the Transformer **encoder**, whose self-attention is naturally bidirectional (no causal mask), and design a pre-training task that *forces* the model to use context from **both sides simultaneously**. That task is masked language modelling.

6.3 Complete architecture



Each encoder block is exactly the one from Chapter 5: multi-head **bidirectional** self-attention → add & norm → feedforward → add & norm.

The input representation

BERT sums **three** embeddings for every token:

Embedding	Purpose
Token embedding	The word/subword itself (WordPiece tokenization).
Segment embedding	Marks whether a token belongs to sentence A or sentence B (for sentence-pair tasks).
Position embedding	Encodes position (learned, not sinusoidal).

Special tokens

- **[CLS]** — prepended to every input; its final vector is used as an aggregate representation of the whole sequence for classification.
- **[SEP]** — separates two sentences (and marks the end).
- **[MASK]** — the placeholder used by the masked language modelling task.

6.4 Pre-training task 1 — Masked Language Modelling (MLM)

This is the heart of BERT. Randomly **mask 15%** of the input tokens and train the model to **predict the original token** from the surrounding context on **both sides**.



Because self-attention sees the whole sentence, predicting the masked word forces the model to fuse left and right context — this is what makes BERT **deeply bidirectional**.

To avoid a mismatch (the **[MASK]** token never appears at fine-tuning time), of the chosen 15%:

- 80% are replaced with **[MASK]**,
- 10% are replaced with a **random** word,
- 10% are **left unchanged**.

The loss is cross-entropy computed **only on the masked positions**.

6.5 Pre-training task 2 — Next Sentence Prediction (NSP)

To help with tasks about **relationships between two sentences** (question answering, entailment), BERT is also trained to predict whether sentence B actually **follows** sentence A.

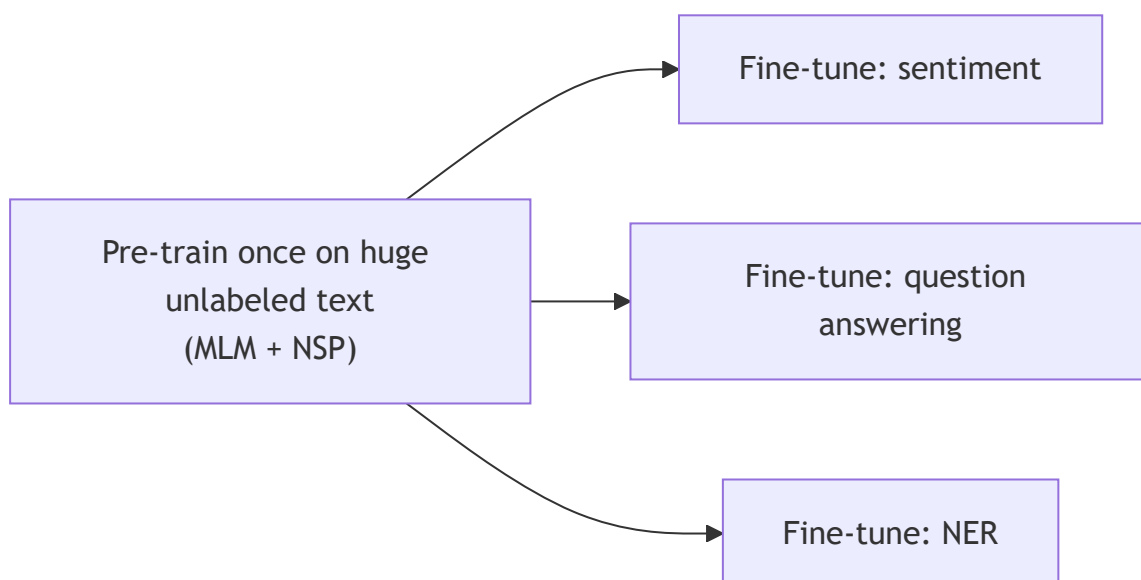
- 50% of the time B is the true next sentence (label **IsNext**).
- 50% of the time B is a random sentence (label **NotNext**).

The **[CLS]** token's final vector feeds a small classifier for this binary decision.

(Later research — e.g. RoBERTa — found NSP to be largely unnecessary and dropped it, but it was part of the original design.)

6.6 The pre-train → fine-tune paradigm

This two-stage recipe is BERT's lasting contribution.



1. **Pre-train** the full encoder once, expensively, on billions of words (BooksCorpus + English Wikipedia). This learns general language.
2. **Fine-tune** by attaching a tiny task-specific head and training briefly on a small labeled dataset. All BERT weights are updated, but cheaply.

How different tasks attach a head:

Task type	What is used
Single-sentence classification (e.g. sentiment)	The [CLS] vector → linear classifier.
Sentence-pair classification (e.g. entailment)	[CLS] vector over the A [SEP] B input.
Token classification (e.g. NER)	Each token's final vector → per-token classifier.
Question answering (SQuAD)	Predict start/end span positions over the passage tokens.

This paradigm meant practitioners no longer needed to design and train a bespoke architecture per task — a single pre-trained model plus a light head reached state-of-the-art across the board.

6.7 Limitations

Limitation	Explanation
Cannot generate text naturally	It is an encoder trained to fill blanks, not to produce fluent left-to-right sequences. Not a language generator.
Pre-train/fine-tune mismatch	<code>[MASK]</code> appears in pre-training but never at inference/fine-tuning.
Only 15% of tokens give signal	Each MLM example trains on a small fraction of positions → sample-inefficient, needs lots of data.
Fixed length & quadratic attention	Inherits the Transformer's $O(n^2)$ cost and a capped context (512 tokens).
Needs fine-tuning per task	Out of the box it cannot follow instructions or do a task zero-shot; each task needs labeled data and a training run.

That last point is the key philosophical contrast with the model in the next chapter.

6.8 How it relates to the next model

BERT took the Transformer **encoder** and optimized for **understanding** via bidirectional masked prediction. Its sibling took the Transformer **decoder** and optimized for **generation** via left-to-right next-word prediction — and, by scaling that idea enormously, discovered that a single generative model could perform many tasks **without task-specific fine-tuning at all**.

That model family is **GPT**.

➡ Continue to [Chapter 7 — GPT](#)

Chapter 7 — GPT

GPT = **G**enerative **P**re-trained **T**ransformer (OpenAI, 2018 onward).

7.1 What it is

GPT is a **stack of Transformer decoders** (the encoder and cross-attention are discarded) trained on a single, simple objective: **predict the next token** given all previous tokens. It is an **autoregressive language model** — it generates text one token at a time, each new token conditioned on everything generated so far.

Where BERT reads and understands, GPT **writes**. And by scaling this simple idea to enormous models and datasets, GPT became a general-purpose system that can perform many tasks from a natural-language prompt alone.

The lineage:

Model	Year	Parameters	Headline idea
GPT-1	2018	117M	Generative pre-training + fine-tuning works.
GPT-2	2019	1.5B	Scale enables zero-shot task performance.
GPT-3	2020	175B	In-context / few-shot learning from the prompt.
GPT-3.5 / ChatGPT / GPT-4	2022+	—	Instruction tuning + RLHF → aligned assistants.

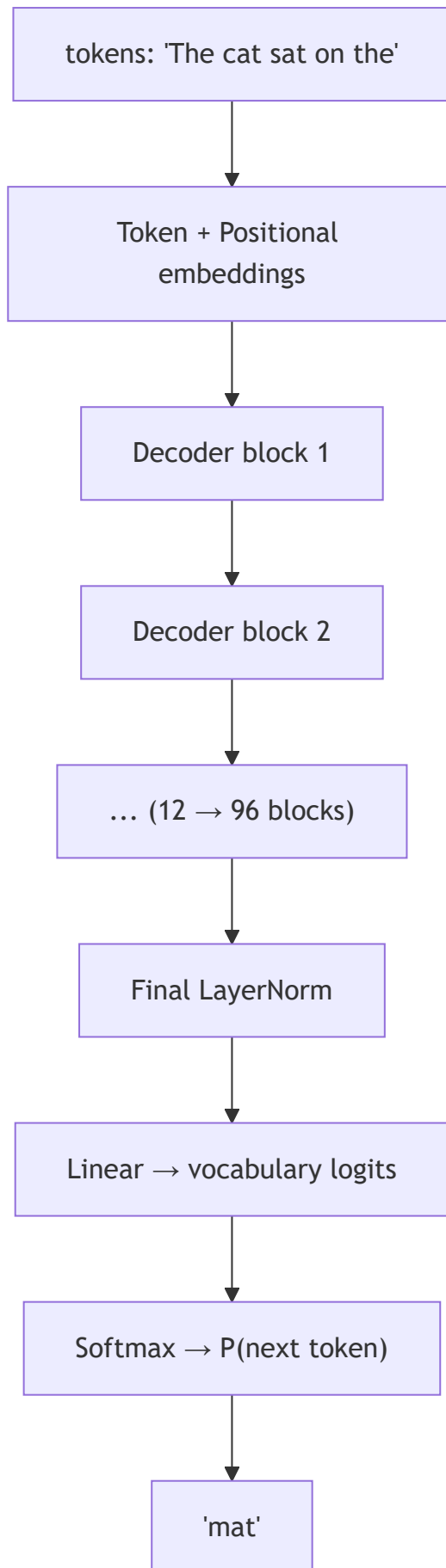
7.2 Why it appeared (the limitation it fixed)

BERT is bidirectional and superb at *understanding*, but it **cannot generate** coherent text, and it **requires task-specific fine-tuning** with labeled data for every new task.

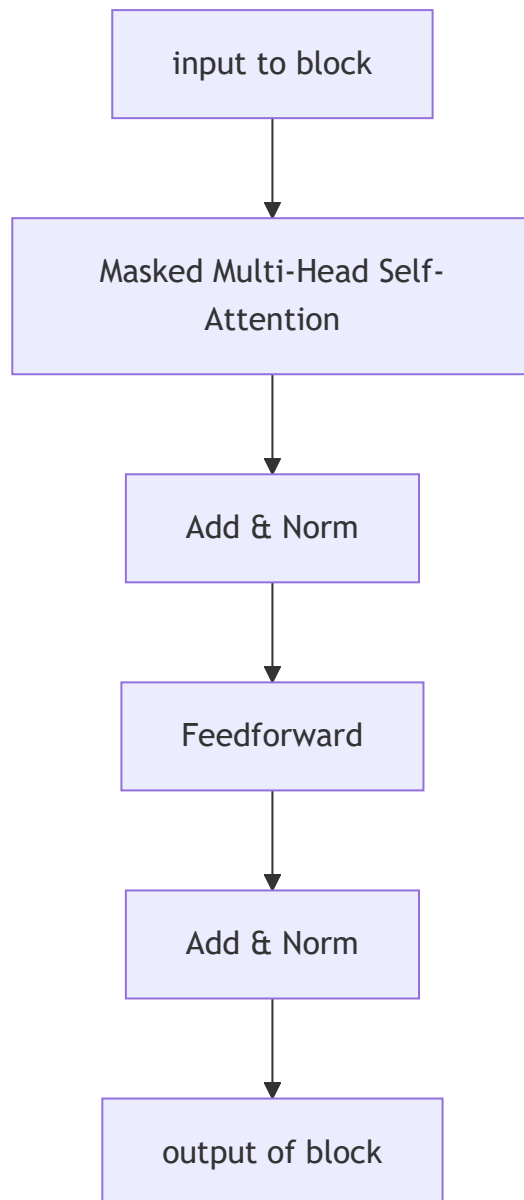
GPT takes the opposite, generation-first stance:

- Use the **decoder** with **causal masking** so the model is a proper left-to-right generator — it can produce fluent, arbitrarily long text.
 - Train on plain next-word prediction, which needs **no labels at all** — any text is training data.
 - Discover (at scale) that a single generative model can do translation, summarization, Q&A, and more **just by being prompted**, removing the need to fine-tune per task.
-

7.3 Complete architecture



Each **decoder block** is a trimmed Transformer decoder — there is no encoder, so the **cross-attention sub-layer is removed**. What remains:



The critical component — masked (causal) self-attention

This is what makes GPT a valid generator. When computing the representation of token i , attention is allowed to look only at tokens $1 \dots i$, never at future tokens. Future scores are set to $-\infty$ before the softmax:

$$\text{mask}_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

Contrast with BERT: BERT's encoder self-attention is **unmasked** (sees both sides); GPT's decoder self-attention is **masked** (sees only the past). This one difference is the whole distinction between the two branches of the Transformer family.

7.4 How it does language modelling

GPT directly optimizes the autoregressive factorization from Chapter 0:

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

The training objective is to maximize the log-likelihood of the next token across a huge corpus:

$$L = \sum_t \log P(w_t \mid w_{<t}; \theta)$$

Data flow for one step: the token sequence is embedded, passed through the masked decoder stack, and the final state at the last position is projected to vocabulary logits and softmaxed to give $P(\text{next token})$.

Generation repeats this: predict a token, append it to the input, predict the next, and so on. Sampling strategies control the style of output:

Strategy	Effect
Greedy	Always take the highest-probability token (deterministic, can be dull).
Temperature	Scales logits before softmax; higher = more random/creative.
Top-k / Top-p (nucleus)	Sample only from the most probable tokens, balancing coherence and diversity.

7.5 The paradigm shift — from fine-tuning to prompting

The most consequential discovery of the GPT line is that **scale changes how you use the model**.

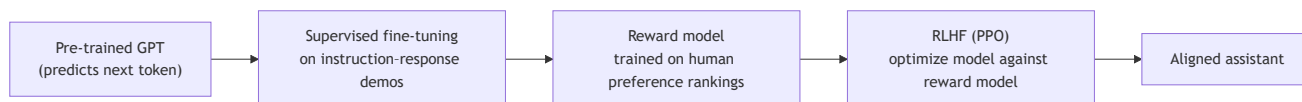
- **GPT-1** still followed BERT's recipe: pre-train, then fine-tune per task.
- **GPT-2** showed that a big enough model can do tasks **zero-shot** — just describe the task in text.
- **GPT-3** demonstrated **in-context (few-shot) learning**: put a few examples *in the prompt* and the model infers the pattern, **without any weight updates**.

```
Prompt:
English: sea otter   → French: loutre de mer
English: cheese      → French: fromage
English: hello       → French:
Model completes: bonjour
```

Nothing was retrained — the model learned the task from the context window at inference time. This is qualitatively new and only emerges at large scale.

7.6 From raw model to assistant — instruction tuning and RLHF

A raw next-token predictor is not automatically helpful or safe; it just continues text. Turning GPT-3 into ChatGPT added two alignment stages:



1. **Supervised fine-tuning (SFT):** train on human-written examples of following instructions, so the model responds to requests rather than merely continuing text.
2. **Reward modelling:** humans rank multiple model outputs; a reward model learns to predict those preferences.
3. **RLHF (Reinforcement Learning from Human Feedback):** the model is optimized (e.g. with PPO) to produce outputs the reward model scores highly — making it more helpful, honest, and harmless.

7.7 Limitations

Limitation	Explanation
Hallucination	Being a fluent next-token predictor, it can state false information confidently.
Unidirectional	Only left context per position; for pure understanding/tagging tasks, bidirectional encoders can be stronger.
Quadratic attention & finite context	Inherits $O(n^2)$ cost; limited (though growing) context window.
Enormous cost	Training and serving very large models is extremely expensive.
No true grounding/reasoning guarantees	Statistical pattern completion, not verified logic; can fail at multi-step reasoning or arithmetic.
Static knowledge	Knowledge is frozen at training time unless augmented (e.g. retrieval/tools).

7.8 How it gave rise to what comes next

GPT established the dominant modern recipe — **a decoder-only Transformer, pre-trained autoregressively at scale, then aligned**. The frontier since then has been about pushing each axis of

this recipe: more scale (with predictable **scaling laws**), longer context, better alignment, multimodality, and efficiency. That is the subject of the final chapter.

 Continue to [Chapter 8 — Beyond](#)

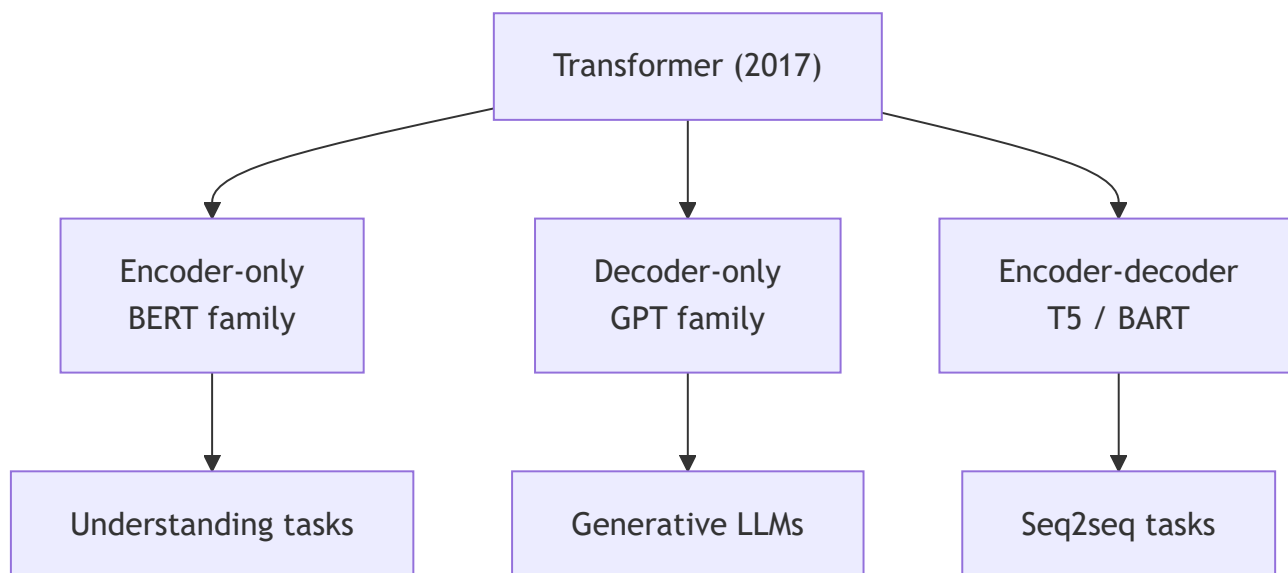
Chapter 8 — Beyond: Modern LLMs

The models in Chapters 6–7 defined two branches of the Transformer family. This chapter places them side by side and surveys the ideas that followed, so the evolution feels complete.

8.1 The three Transformer families

Every large model today is one of three configurations of the Transformer:

Family	Blocks used	Attention	Best at	Examples
Encoder-only	Encoder	Bidirectional	Understanding: classification, NER, retrieval	BERT, RoBERTa, DeBERTa
Decoder-only	Decoder	Causal (masked)	Generation, general-purpose prompting	GPT, LLaMA, Mistral, Claude, Gemini
Encoder-decoder	Both	Bi + cross	Seq-to-seq: translation, summarization	T5, BART



8.2 Key models and ideas after BERT/GPT

- **RoBERTa (2019)** — BERT trained better: more data, longer, larger batches, and NSP removed. Showed BERT was under-trained; recipe matters as much as architecture.

- **T5 (2019)** — "Text-to-Text Transfer Transformer." Casts **every** NLP task as text→text ("translate English to German: ..."), unifying tasks under one encoder–decoder model and objective.
- **BART (2019)** — Encoder–decoder trained as a denoising autoencoder; strong at generation tasks like summarization.
- **GPT-3 (2020)** — Few-shot in-context learning at 175B parameters (Chapter 7).
- **Scaling laws (Kaplan 2020; Chinchilla 2022)** — Performance improves predictably with model size, data, and compute. Chinchilla showed most large models were **under-trained on data**: for a fixed compute budget, smaller models trained on more tokens win.
- **InstructGPT / ChatGPT (2022)** — Instruction tuning + RLHF turned raw LLMs into usable assistants (Chapter 7).
- **LLaMA / Mistral / Falcon (2023+)** — Efficient, open-weight decoder-only models; strong performance at smaller sizes, democratizing LLMs.
- **GPT-4, Claude, Gemini (2023+)** — Frontier multimodal models (text + images and more), stronger reasoning, much longer context.

8.3 Architectural refinements you will encounter

Modern decoder-only LLMs keep the Chapter 5 skeleton but swap in improved components:

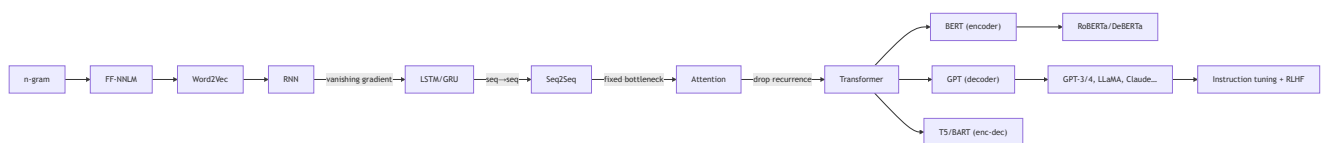
Refinement	Replaces	Why
Rotary Positional Embeddings (RoPE)	Sinusoidal/learned positions	Better relative-position modelling and length extrapolation.
RMSNorm	LayerNorm	Cheaper normalization with equal or better stability.
SwiGLU / GeGLU	ReLU FFN	Gated activations improve quality.
Grouped-Query Attention (GQA)	Multi-head attention	Fewer key/value heads → faster, cheaper inference.
FlashAttention	Naive attention kernel	Exact attention computed with far less memory traffic.
Mixture-of-Experts (MoE)	Dense FFN	Route each token to a few "expert" sub-networks → more capacity at similar compute.

8.4 Tackling the quadratic-attention limitation

The $O(n^2)$ cost of self-attention is the main barrier to long context. Directions:

- **Efficient/sparse attention** (Longformer, BigBird): attend to a local window plus a few global tokens → near-linear cost.
- **Linear attention / state-space models** (Mamba): reformulate sequence mixing to scale linearly with length; a partial return of recurrence-like ideas, but parallelizable.
- **Retrieval-augmented generation (RAG)**: fetch relevant documents at query time and put them in the context, extending effective knowledge without enlarging the model.

8.5 The full evolution, at a glance



8.6 The recurring pattern of every leap

Reading the whole guide, one pattern repeats at every step:

1. A model works but has a **specific structural limitation**.
2. The next model **isolates that limitation** and redesigns exactly that part — keeping what worked.

From → To	Limitation removed
n-gram → NNLM	No word similarity; sparse counts
NNLM → RNN	Fixed context window
RNN → LSTM	Vanishing gradients / short memory
LSTM → Seq2Seq	Can't map different-length sequences
Seq2Seq → Attention	Single fixed-vector bottleneck
Attention → Transformer	Sequential, non-parallel recurrence
Transformer → BERT	No bidirectional pre-training for understanding
Transformer → GPT	No scalable autoregressive generation / prompting
GPT → aligned LLMs	Not helpful/safe out of the box

Understanding *why* each arrow exists is the real goal of this guide — architectures will keep changing, but this problem-then-fix reasoning is what lets you understand whatever comes next.

8.7 A compact glossary

- **Autoregressive** — generates output one token at a time, each conditioned on the previous ones.
- **Context vector** — a vector summarizing input, passed from encoder to decoder.
- **Attention weights** — normalized relevance scores deciding how much each position contributes.
- **Query / Key / Value** — the three projections underlying attention (search term / index label / stored content).
- **Self-attention** — attention of a sequence to itself.
- **Causal mask** — prevents a position from attending to future positions.
- **Pre-training / fine-tuning** — learn general language once, then specialize cheaply.
- **In-context learning** — learning a task from examples in the prompt, with no weight updates.
- **RLHF** — aligning a model to human preferences via a learned reward signal.

 [Back to the guide index](#)

The Evolution of Language Models — A Complete Study Guide

This guide walks through the evolution of neural language models in the order in which each idea solved a limitation of the previous one. Every chapter follows the same structure so you can build a clear mental model:

- 1. **What it is** — the core idea in plain language.
- 2. **Why it appeared** — the limitation of the previous model it fixes.
- 3. **Complete architecture** — a diagram plus every component explained.
- 4. **How it does language modelling** — the math and the data flow, step by step.
- 5. **Training** — loss functions and how learning happens.
- 6. **Limitations** — what it still cannot do well.
- 7. **How it gave rise to the next model** — the bridge to the following chapter.

Reading order

#	Chapter	Key idea introduced
0	Foundations	What a language model is; n-grams → neural LMs → word embeddings
1	RNN	A hidden state that carries memory across a sequence
2	LSTM & GRU	Gates that preserve long-range memory
3	Seq2Seq	Encoder–decoder for mapping one sequence to another
4	Attention	Let the decoder look back at all encoder states
5	Transformer	Self-attention only — remove recurrence entirely
6	BERT	Bidirectional pre-training (encoder-only)
7	GPT	Autoregressive pre-training (decoder-only)
8	Beyond	Scaling, T5, instruction tuning, RLHF, modern LLMs

The one-picture summary



The single thread that connects everything

Every model in this guide is trying to answer one question well:

Given some context of words, what is the probability of the next (or a missing) word?

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

The entire history below is a series of better and better ways to model that conditional probability $P(w_t \mid \text{context})$ — first with counts, then with recurrence, then with gates, then with attention, and finally with pure self-attention at massive scale.